

Universidad Nacional Autónoma de México

FACULTAD DE CIENCIAS, 2027-I

COMPILADORES



PRÁCTICA 02

ANALIZADOR LÉXICO COMPLETO DE MINIC

EQUIPO:

Delgado Díaz Hermes Alberto - 319258613

Medina Peralta Joaquín - 320202513

21 de septiembre de 2026

1. Objetivo

El objetivo de esta práctica es el conocer e implementar un analizador sintáctico descendente recursivo, es decir, un parser para el lenguaje MiniC que tiene como rol el consumir la secuencia de tokens generados por el analizador léxico, verificando si los elementos se encuentran correctos con respecto a las producciones de la gramática.

Su papel principal es el conectar el analizador léxico con la construcción futura del AST, que ayuda al validar el orden y la agrupación de los tokens cumpla con la gramática para posteriormente poder construir su representación.

2. Evolución respecto de la Práctica 2

2.1. Módulos modificados o agregados

Para esto, se incorpora los componentes del analizador sintáctico por medio de la carpeta de `parser/` dentro de la estructura del proyecto, generando los archivos `parser.h`, `parser.c`. De este modo, el archivo `main.c` fue ampliado para que se pueda coordinar el lexer y el parser del compilador.

2.2. Interfaz incremental incorporada al lexer

Para este momento del proyecto, se cambia la funcionalidad del analizador léxico para dejar de procesar la entrada por bloque y se usa una interfaz incremental, donde cada vez que el parser realiza una lectura del token del archivo, el lexer procesa ese único token disponible para conservar el estado interno y de ese punto continuar con la siguiente llamada del parser.

2.3. Eliminación de la impresión de tokens como efecto secundario

Durante la práctica pasada, el lexer imprimía los tokens en la consola para su verificación pero en estos momentos, la salida fue eliminada para que los tokens sean consumidos como entrada del parser.

2.4. Refactorizaciones realizadas

Para este caso, las principales refactorizaciones del proyecto fue en la implementación de `lexer.c`, al agregar una nueva interfaz en el `lexer.h` para indicar el estado del lexer, si hay error en la entrada o salida del programa y si hay errores de memoria. Además de agregar una estructura para el lexer, dejando el archivo a leer, la columna y fila actuales, y si se encontro el final del archivo.

A su vez, el ciclo principal del lexer se paso en su nueva función de `lexer_next_token` como la destrucción del lexer para la limpieza en memoria.

2.5. Funcionalidades léxicas conservadas

El sistema principal de nuestro lexer se mantuvo intacto ante las nuevas exigencias de esta práctica, el manejo y lectura del archivo fuente, el sistema de almacenamiento de tokens, el seguimiento de línea y columna, la gestión de espacios en blanco y comentarios, el reporte de errores léxicos, el reconocimiento de identificadores, palabras reservadas, literales y operadores.

3. Arquitectura e interfaz

Para esto, el analizador sintáctico está estructurado bajo el paradigma de descenso recursivo, teniendo su lógica en funciones individuales que son traducciones de las producciones, la cual se encuentra encapsulada en su propio módulos y con una cabecera pública que expone sus funciones principales y la estructura de estado del parser.

En esta misma cabecera, se dejó lista la estructura para representar el estado actuales del parser que mantiene un puntero hacia el analizador léxico instanciado, teniendo banderas de control como la gestión de errores (como si hay un error o el modo pánico), el token actual y el anterior teniendo como función la anticipación de tokens para las decisiones futuras del parser.

De esta manera, podemos definir las responsabilidades del lexer, parser y `main.c` como:

- El lexer tiene el rol de leer el archivo y proporcionar tokens válidos o inválidos bajo las peticiones del parser, sin el almacenamiento de los mismos.
- El parser se encarga de solicitar los tokens, verificar que su secuencia cumpla con la gramática, reportar fallas y tener un mecanismo de sincronización ante errores.
- Por último, el main es el orquestador de ambas fases, validando los argumentos de entrada, inicializando el lexer y el parser, hace el análisis global, reporta el resultado y libera los recursos antes de la salida del programa.

Con respecto a la liberación de memoria, durante la ejecución del parser al solicitar tokens al lexer, adquiere el manejo absoluto de la memoria asociada al lexema, lo cual durante su avance el parser se encarga de liberar la memoria del token anterior llamando la función de

`token_destroy(&parser->previous)` antes de escribir el actual. Por último del programa se llama la función `parser_destroy` auxiliar para limpiar cualquier token remanente del estado.

Finalmente, para tratar los fallos internos del lexer, se utiliza la estructura de estado del lexer, ya que nuestra función de `parser_advance` intercepta este estado y detiene el ciclo de advance para informar en la salida de errores. Además de que, el analizador léxico puede devolver el token de error y en este caso, el parser imprime el diagnóstico léxico, marca como inválido, destruye el token y continúa al siguiente token válido.

4. Descenso recursivo y gramática

La técnica que se usa, establece que el mapeo directo de un uno a uno entre los símbolos no terminales de nuestra gramática, la cual fue mapeada directamente a su función dentro del parser, para validar su propia estructura mediante el consumo de los tokens y la invocación de otras producciones.

De este modo, nuestra función para revisar la regla gramatical dentro de `parse_statement` tiene que inspeccionar el token actual, de ahí se usa un switch para enrutar el control hacia alguna función adecuada siguiendo las reglas gramaticales.

5. Expresiones

La gramática de expresiones se implementa mediante descenso recursivo por niveles, donde cada nivel de precedencia cuenta con su propia función. El análisis inicia en el nivel de menor prioridad y escala por medio de llamadas secuenciales hacia los niveles superiores, verificando que las operaciones con mayor precedencia se agrupen primero. La asociatividad por la izquierda para los operadores binarios se maneja iterando con bucles while, mientras que la asociatividad por la derecha del operador unario negativo se logra mediante recursión directa sobre su propia rama. Finalmente, la función base `parse_primary` consume los símbolos terminales y procesa las agrupaciones, generando un nuevo ciclo de evaluaciones,

6. Diagnósticos y recuperación

Para el diagnóstico de los errores sintácticos es por medio de emitir cuando el token de anticipación no coincide con la estructura gramatical esperada por la regla actual, imprimiendo la posición, el elemento esperado y el lexema encontrado.

Se diferencia de un token léxico `ERROR` en que este último proviene de un fallo de escaneo del lexer y es consumido de inmediato registrando la falla léxica, evitando emitir un error sintáctico redundante.

Para recuperarse de un fallo, el parser activa el modo pánico y sincroniza avanzando hasta encontrar puntos seguros como delimitadores, inicios de sentencia o el final del archivo. El progreso del análisis y la prevención de ciclos infinitos se garantizan forzando el avance sobre tokens no reconocidos o devolviendo el control a la estructura superior en límites de bloque.

Finalmente, la validación completa del archivo se verifica en `parse_program`, la cual exige el consumo explícito de `TOKEN_EOF` tras procesar la lista de sentencias para asegurar que no quede contenido extra sin analizar.

7. Resultados y pruebas

Para las pruebas, se ejecutaron los tests públicos del proyecto para esta fase del compilador donde se pudo observar que todos pasaron sin problemas. Además de que se agregaron las pruebas de regresión de las prácticas pasadas, dejándolas aparte de su ejecución en otra regla del archivo `Makefile`.

A continuación se puede observar los logs de ambas pruebas:

```
> make test
[OK] p01_empty
[OK] p02_declarations
[OK] p03_assignment_print
[OK] p04_expression_precedence
[OK] p05_if_else
[OK] p06_dangling_else
[OK] p07_while_blocks
[OK] p08_complete_program
[OK] p09_missing_semicolon
[OK] p10_missing_parenthesis
[OK] p11_missing_brace
[OK] p12_incomplete_expression
[OK] p13_operator_without_operand
[OK] p14_unexpected_token
[OK] p17_trailing_content
[OK] p15_multiple_errors
[OK] p16_lexical_error
```

Aprobadas: 17

Fallidas: 0

```
> make test-regression
[OK] p01_identifiers
[OK] p02_reserved_words
[OK] p03_case_sensitive
[OK] p04_booleans
[OK] p05_compound_operators
[OK] p06_incomplete_operators
[OK] p07_comments
[OK] p08_comment_at_eof
[OK] p09_slash_and_comments
[OK] p10_longest_match
[OK] p10_mixed_basic
[OK] p11_error_recovery
[OK] p12_complete_lexer
[OK] p13_crlf
```

```
[OK] p1_empty
[OK] p2_whitespace
[OK] p3_single_symbols
[OK] p4_integer
[OK] p5_integer_expression
[OK] p6_multiple_lines
[OK] p7_tabs_spaces
[OK] p8_error_character
[OK] p9_negative_number
```

Aprobadas: 23

Fallidas: 0

Además de que se revisó el uso de la memoria con el programa `valgrind`, checando cuanta memoria se ocupó en la ejecución de nuestro compilador y cuanta memoria se liberó, indicando si hubo fuga de memoria. En este caso, se realizó para la prueba con nombre `./tests/public/inputs/p15_multiple_errors.` y se obtuvo lo siguiente:

```
> make memcheck FILE=./tests/public/inputs/p15_multiple_errors.mc
==144845== Memcheck, a memory error detector
==144845== Copyright (C) 2002–2024, and GNU GPL'd, by Julian Seward et al.
==144845== Using Valgrind 3.25.1 and LibVEX; rerun with -h for copyright info
==144845== Command: ./minic ./tests/public/inputs/p15_multiple_errors.mc
==144845==
Error_sintactico [2:0]: se esperaba '; ' al final de la declaracion , pero se encontro 'print'.
Error_sintactico [4:4]: se esperaba una expresion , pero se encontro '; '
.
==144845==
==144845== HEAP SUMMARY:
==144845==    in use at exit: 0 bytes in 0 blocks
==144845==    total heap usage: 26 allocs, 26 frees, 4,708 bytes allocated
==144845==
==144845== All heap blocks were freed — no leaks are possible
==144845==
==144845== For lists of detected and suppressed errors, rerun with: -s
==144845== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

8. Uso de herramientas de Inteligencia Artificial

No se utilizó ninguna herramienta de inteligencia artificial para la elaboración de la práctica.

9. Conclusiones

El desarrollo del analizador sintáctico descendente recursivo de MiniC requirió superar desafíos como ciclos infinitos en el procesamiento, el silenciamiento incorrecto de diagnósticos múltiples y la necesidad de adaptar el entorno de pruebas para capturar la salida de errores. La transición hacia una interfaz incremental en el lexer optimizó el uso de memoria, mientras que la técnica de descenso recursivo por niveles demostró ser altamente efectiva para validar la gramática y gestionar correctamente la precedencia y asociatividad de las expresiones. Finalmente, la exitosa implementación del modo pánico ayudó al compilador a seguir ejecutando para recuperarse de errores y continuar el análisis, consolidando una arquitectura para una futura implementación de AST.

10. Referencias

- Pablo Martínez, Y. J.; Salazar González, P. Y. (2026). Compiladores - Práctica 3, semestre 2027-1. Facultad de Ciencias, UNAM.